Course Code: BTCS601-18	Course Title : Compiler Design	3L:0T:0P	3Credits

Detailed Contents:

UNIT 1: Unit I Introduction to Compilers:
Structure of a compiler – Lexical Analysis – Role of Lexical Analyzer – Input Buffering – Specification of Tokens – Recognition of Tokens – Lex – Finite Automata – Regular Expressions to Automata – Minimizing DFA. **[8 hrs., CO 1]**

Unit II :Syntax Analysis:
Role of Parser – Grammars – Error Handling – Context-free grammars – Writing a grammar, Top-Down Parsing – General Strategies Recursive Descent Parser – Predictive Parser-LL(1) Parser-Shift Reduce Parser-LR Parser-LR (0) Item Construction of SLR Parsing Table - Introduction to LALR Parser – Error Handling and Recovery in Syntax Analyzer-YACC.
[8 hrs., CO 2]

Unit III : Intermediate Code Generation:
Syntax Directed Definitions, Evaluation Orders for Syntax Directed Definitions, Intermediate Languages: Syntax Tree, Three Address Code, Types and Declarations, Translation of Expressions, Type Checking. **[8 hrs., CO 3]**

Unit IV: Run-Time Environment and Code Generation:

Storage Organization, Stack Allocation Space, Access to Non-local Data on the Stack, Heap Management – Issues in Code Generation – Design of a simple Code Generator.
[6 hrs., CO 4]

Unit V: Code Optimization:
Principal Sources of Optimization – Peep-hole optimization – DAG- Optimization of Basic Blocks-Global Data Flow Analysis – Efficient Data Flow Algorithm. **[6 hrs., CO 5]**

Course Outcomes:
After undergoing this course, the students will be able to:

CO1: Build concepts on lexical analysis.

CO2: Understand strategies of syntax analysis.

CO3: Learn techniques of Intermediate code generation.

CO4: Undestand code design issues and design code generator.

CO5: Design and develop optimized codes.

Suggested Readings/ Books:

1. A.V. Aho, Monica, R.Sethi, J.D.Ullman, "Compilers, Principles, Techniques and Tools", Second Edition, Pearson Education/Addison Wesley, 2009.

2. Andrew W. Appel, "Modern Compiler Implementation in Java", Second Edition, 2009.
3. J.P. Tremblay and P.G. Sorrenson, "The Theory and Practice of Compiler Writing", McGraw Hill, 1985.

Course Code: BTCS604-18	Course Title: Compiler Design Lab	L:0;T:0; 2P	1Credits

Sr. No.	No. List of Experiments
1	Design a lexical analyser for given language and the lexical analyser should ignore redundant spaces, tabs and new lines. It should also ignore comments. Although the syntax specification states that identifiers can be arbitrarily long, you may restrict the length to some reasonable value. Simulate the same in C language.
2	Write a C program to identify whether a given line is a comment or not.
3	Write a C program to recognize strings under 'a', 'a*b+', 'abb'.
4	Write a C program to test whether a given identifier is valid or not.
5	Write a C program to simulate lexical analyzer for validating operators.
6	Implement the lexical analyzer using JLex, flex or other lexical analyzer generating tools.
7	Write a C program for implementing the functionalities of predictive parser for the mini language specified in Note 1.
8	a) Write a C program for constructing of LL (1) parsing. b) Write a C program for constructing recursive descent parsing.
9	Write a C program to implement LALR parsing.
10	a) Write a C program to implement operator precedence parsing. b) Write a C program to implement Program semantic rules to calculate the expression that takes an expression with digits, + and * and computes the value.
11	Convert the BNF rules into YACC form and write code to generate abstract syntax tree for the mini language specified in Note 1.
12	Write a C program to generate machine code from abstract syntax tree generated by the parser. The instruction set specified in Note 2 may be considered as the target code.